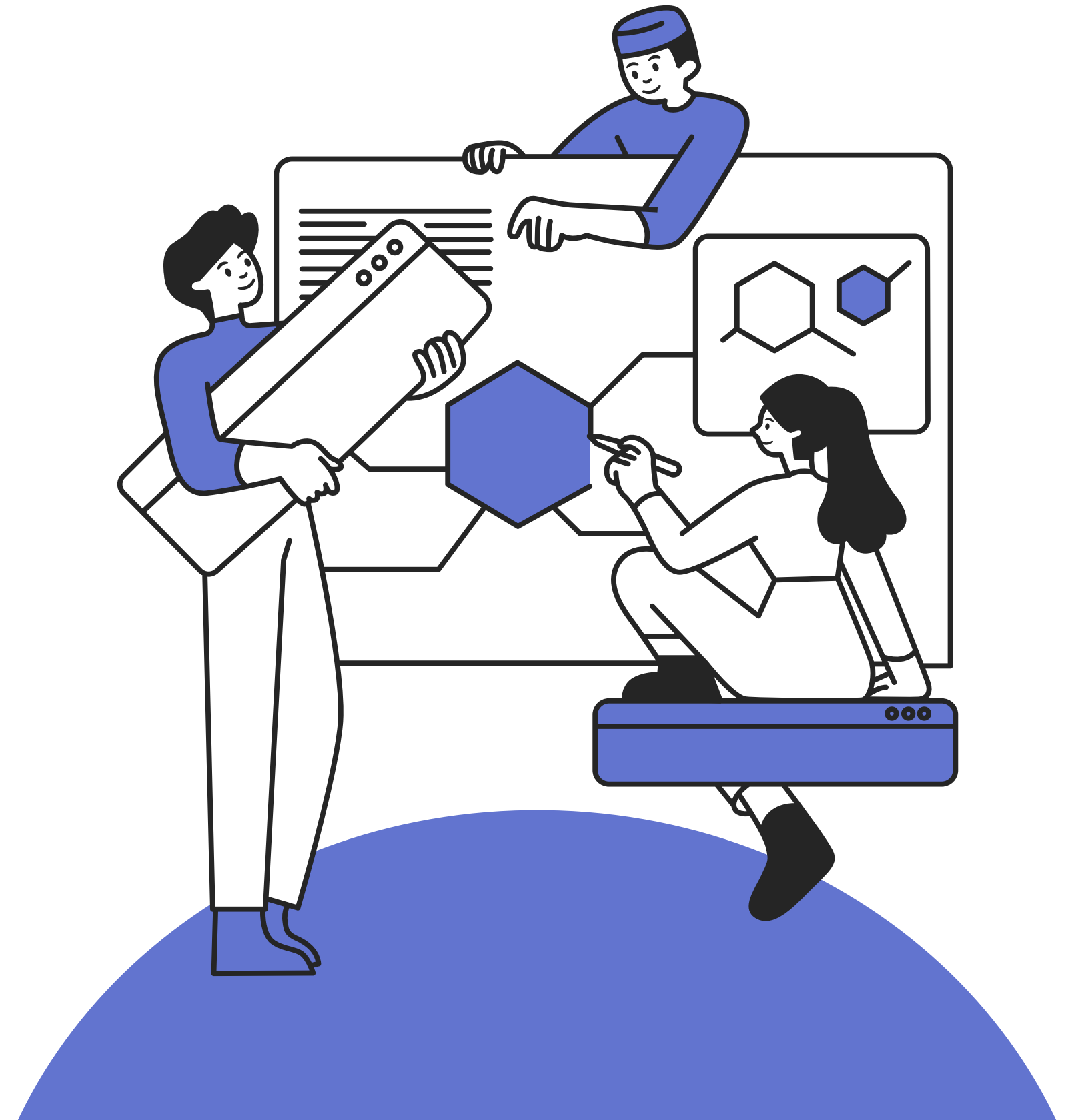
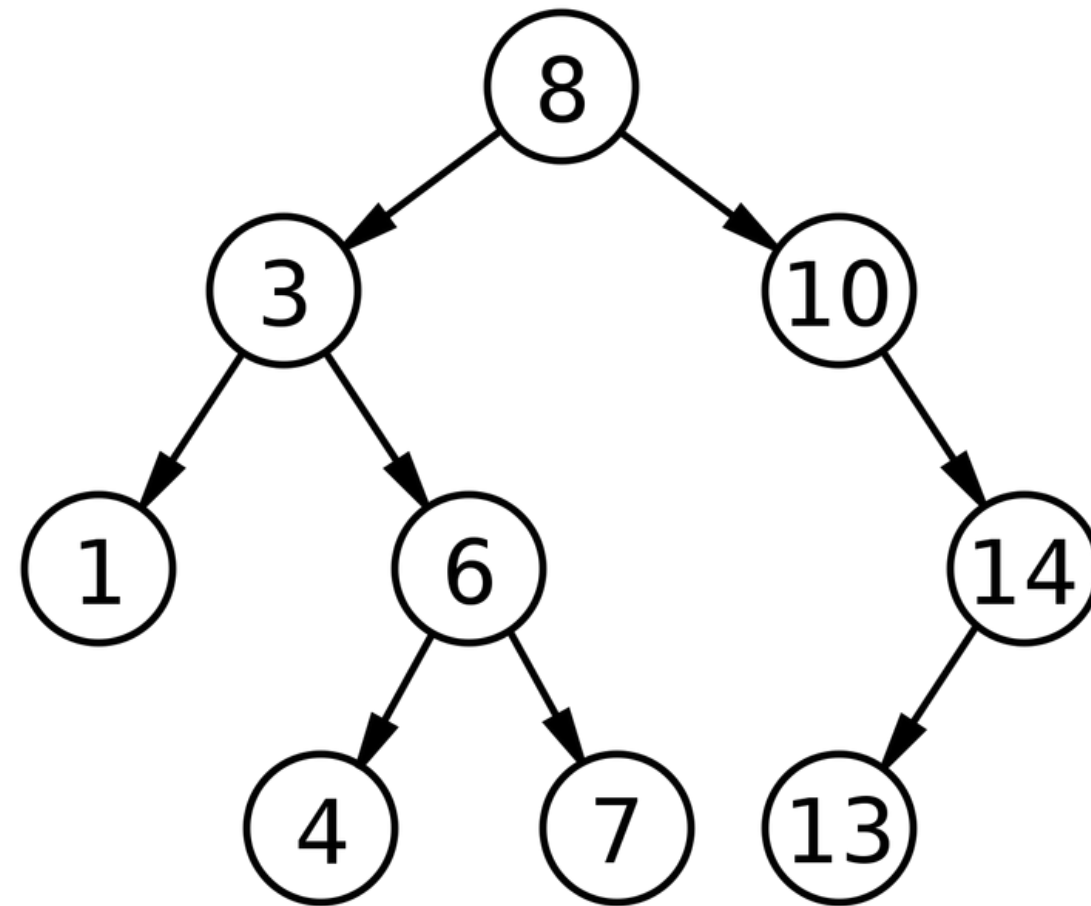


해싱과 해시 테이블

이예린T

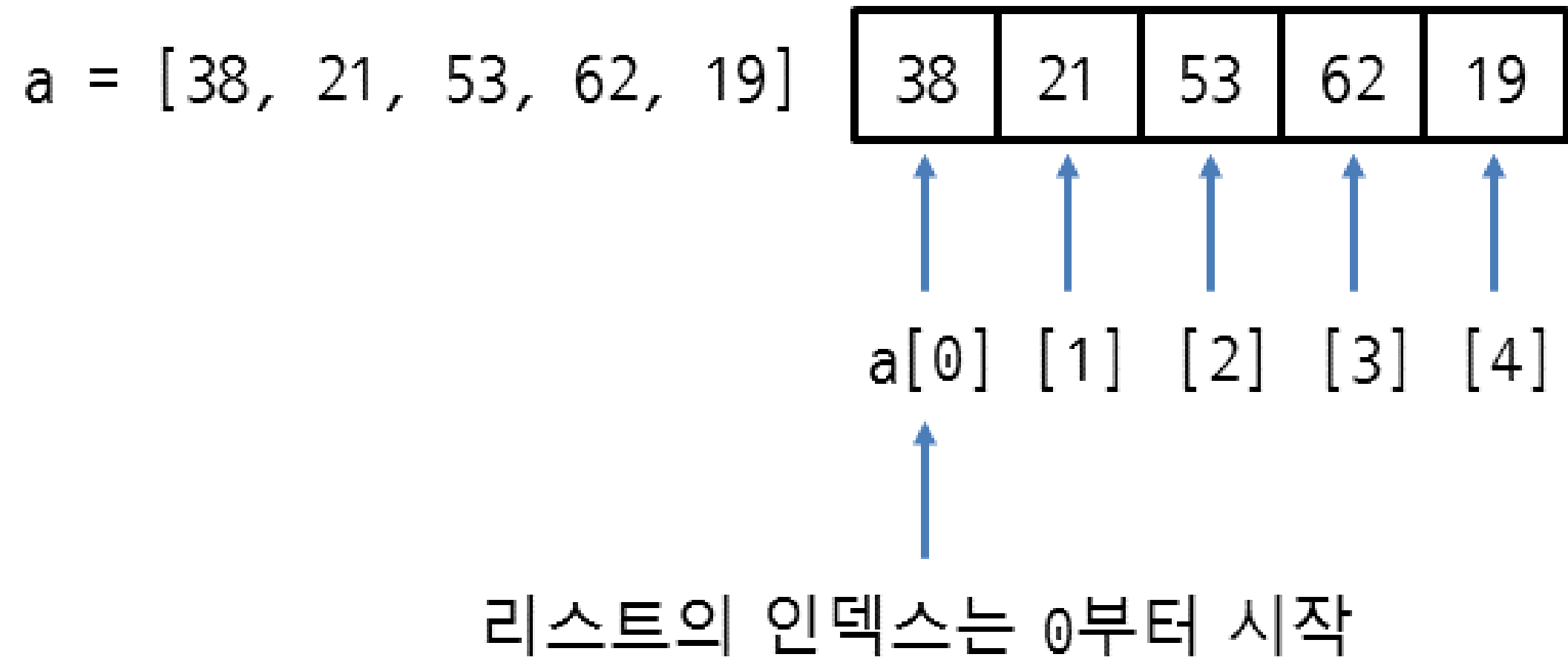


탐색이 가장 빠른 자료구조는??



BST의 탐색 속도:
 $O(\log n)$

그렇다면 배열은?



CONTENTS

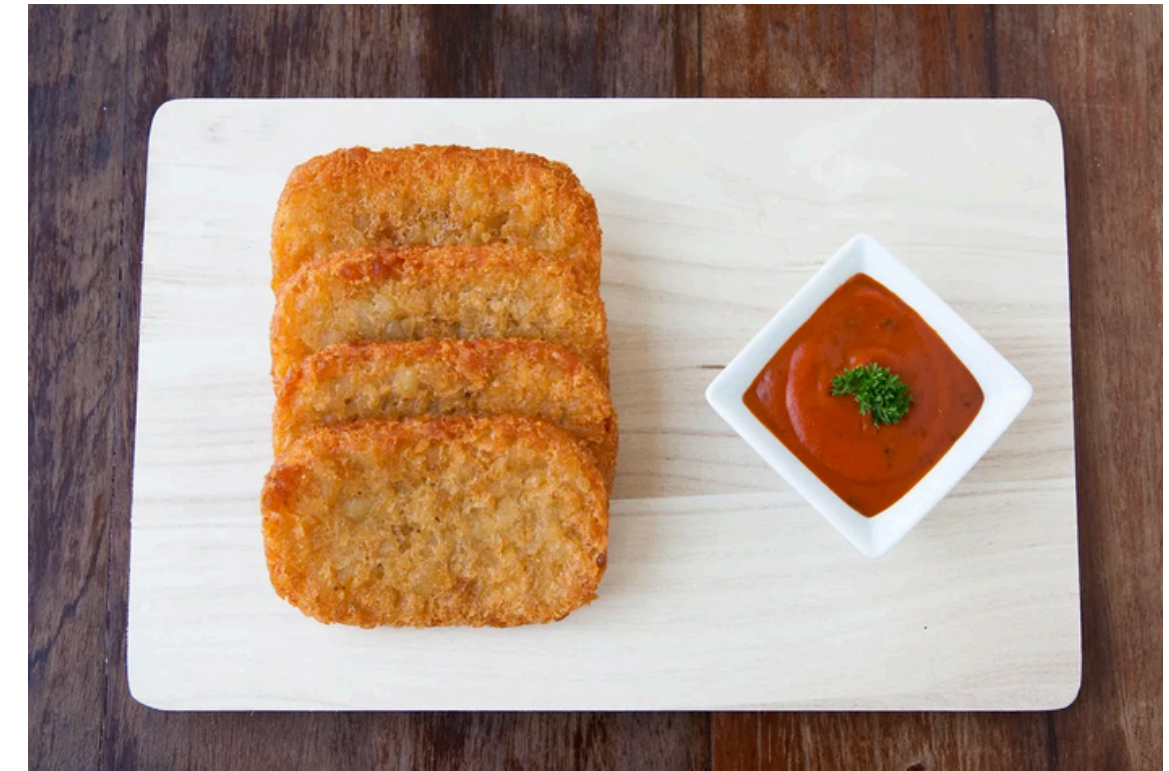
- 1 해싱의 소개
- 2 충돌 해결
- 3 모듈별 탐구
- 4 해싱과 해시 테이블 정리

01

해싱

인간의 욕심은 끝이 없고 ...

- 해시: 고기나 감자같은 재료를 잘게 다져 만든 요리.
- 컴퓨터 과학에서의 해시는 비슷하게 입력 받은 데이터를 처리하고 그 결과로 완전히 달라진 모습의 데이터를 얻는 것
- 해싱: 해시 함수를 이용해 데이터를 변환/저장/검색하는 기법
- 데이터를 담은 테이블을 미리 크게 확보하고, 입력 받은 데이터를 해시 함수로 처리하여 테이블 내 주소를 계산하고 이 주소에 입력한 데이터를 저장
- 공간 효율성을 희생시켰지만, 탐색, 삽입, 삭제 빅오는 $O(1)$

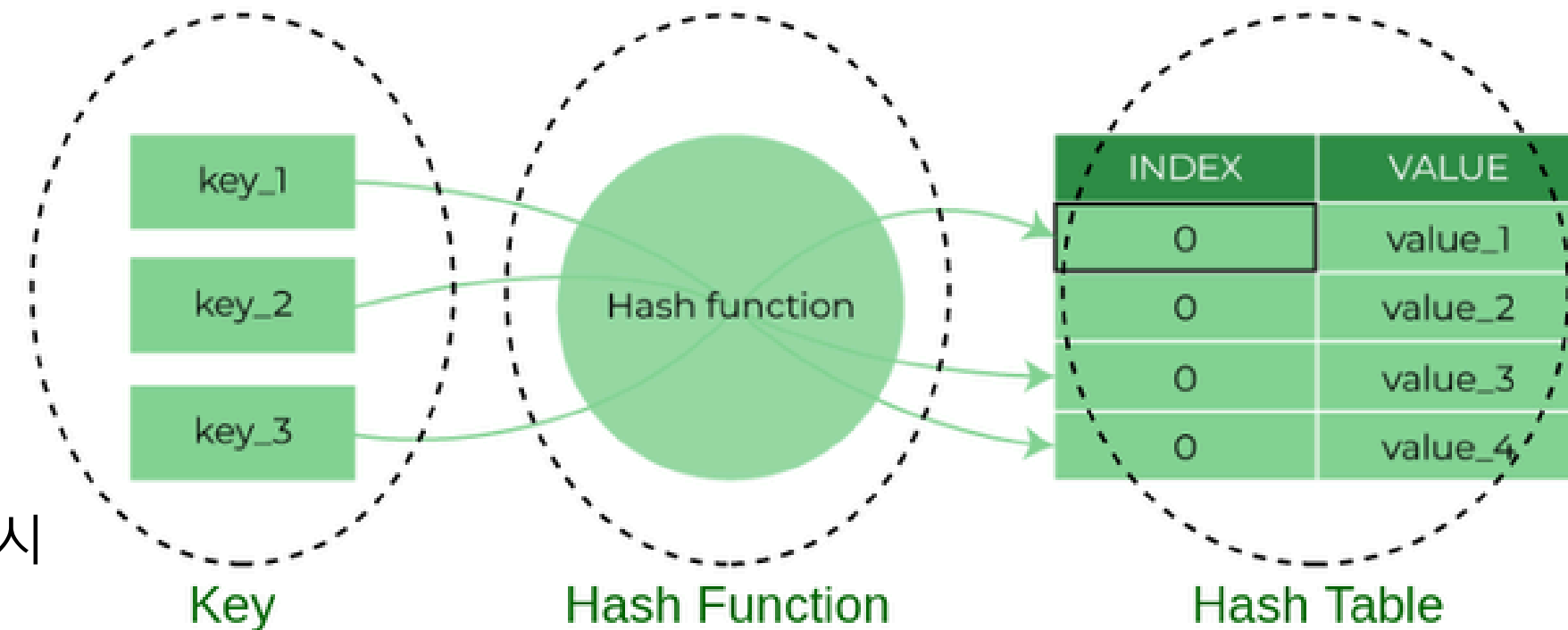


02

해시 테이블과 해시 함수

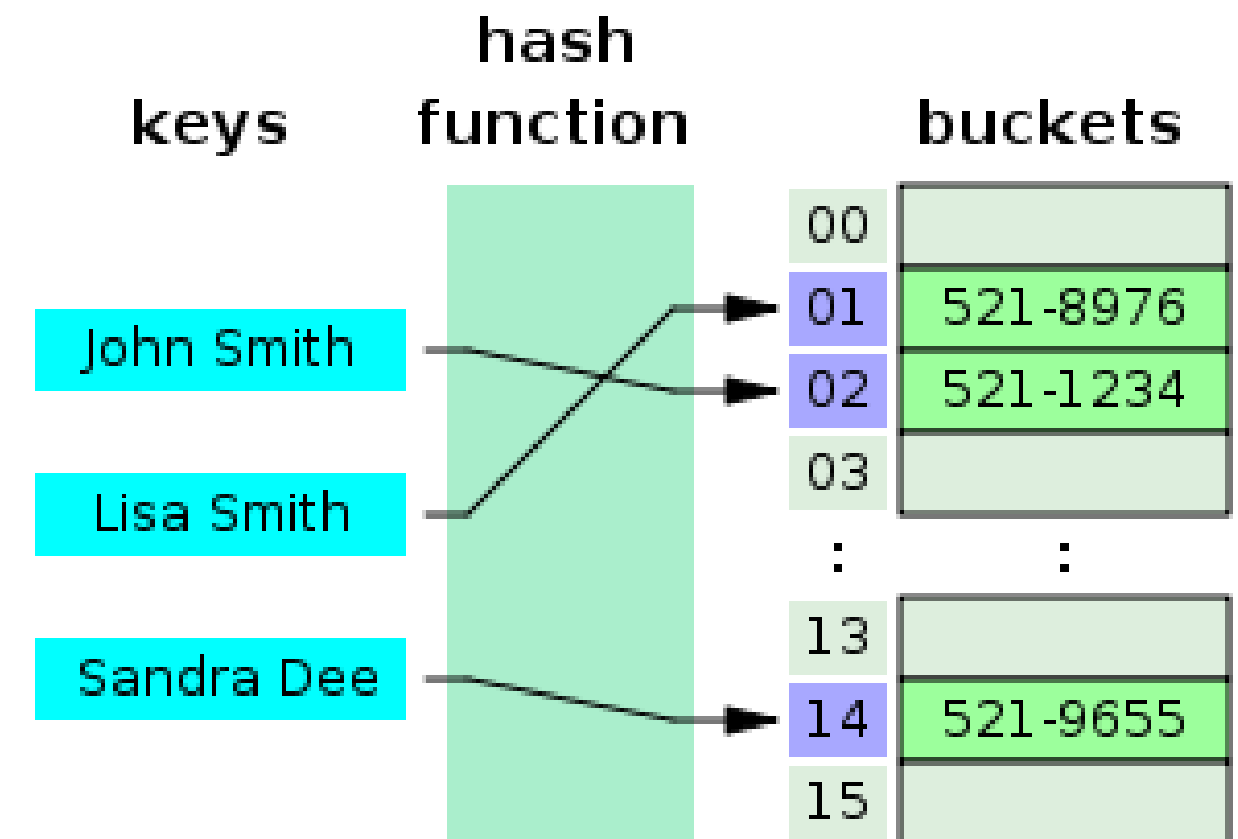
해시 테이블(Hash Table)

- 해싱 기법을 적용해서 만든 구체적인 자료구조
- 데이터의 해시 값을 테이블 내의 주소로 사용하는 자료구조로, 해시 맵(hash map)이라고도 한다.
- 데이터를 담은 테이블을 미리 크게 확보하고, 입력 받은 데이터를 해시 함수로 처리하여 테이블 내 주소를 계산하고 이 주소에 입력한 데이터를 저장하는 것이 해시 테이블이다.



해시 함수(Hash Function)

- 키를 인덱스로 변환하는 함수
- 입력 받은 데이터를 통해 테이블 내의 주소를 계산하는 함수
- 해시 함수를 사용하는 것을 해싱(hashing)이라고 한다.
- 성능이 좋은 해시 함수는 충돌을 최소화하고, 연산이 쉽고 빠르며, 해시 값이 균일한 분포가 되며, 사용할 키의 모든 정보를 이용한다.
- 가장 간단한 해시 함수 알고리즘으로는 나눗셈법, 곱셈법, 자릿수 접기 등이 있다.

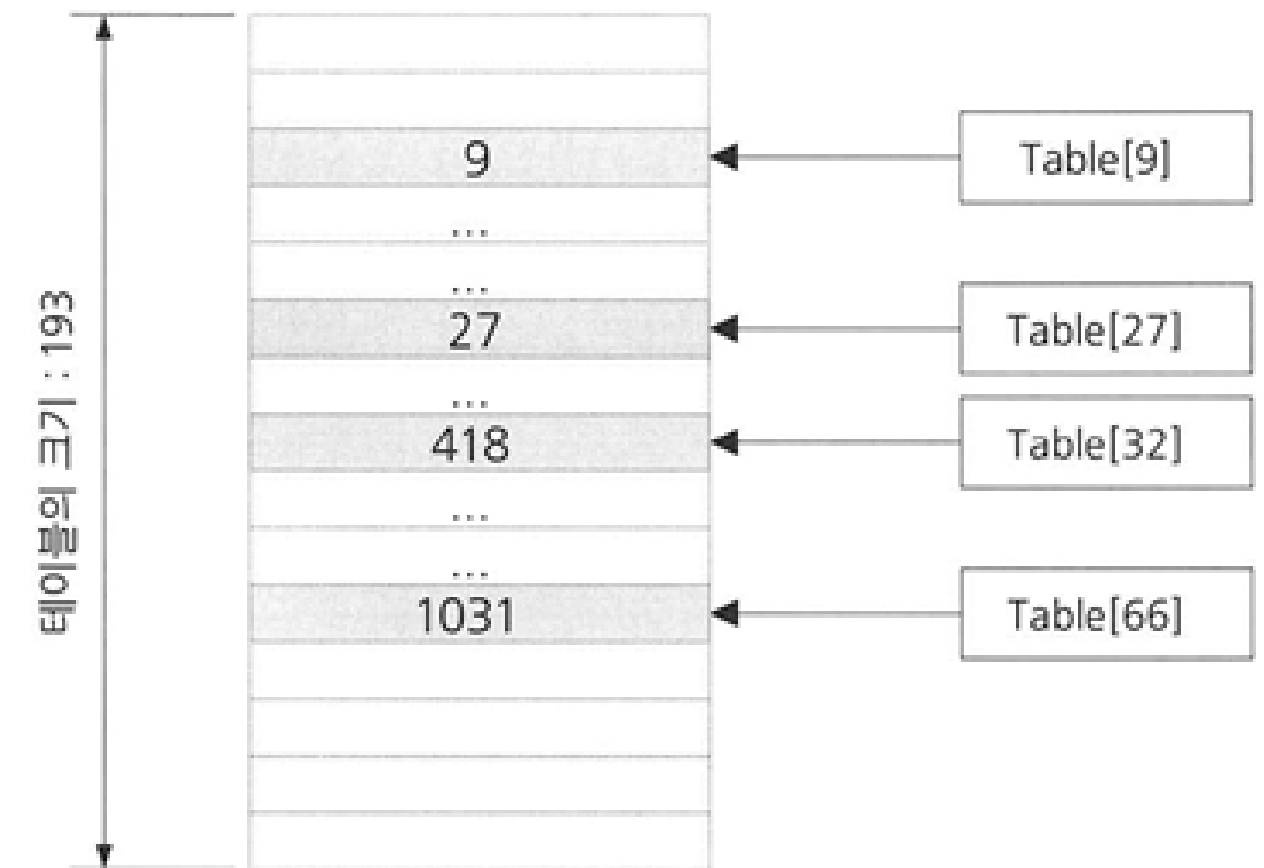


03

해시 함수의 종류

나눗셈법

- 입력 값을 테이블의 크기로 나누고, 그 '나머지'를 테이블의 주소로 사용
- 주소 = 입력값 % 테이블 크기
- 주소 값은 절대 테이블의 크기(n)을 넘지 않고, 0부터 n-1 사이의 주소를 반환함을 보장
- 최적의 n 값은 몇일까?
- 일반적으로 함수가 테이블 공간을 효율적으로 사용하기 위해서는 n을 소수로 정하는 것이 좋고, 특히 2의 제곱수와 거리가 먼 소수를 사용하는 것이 좋다.



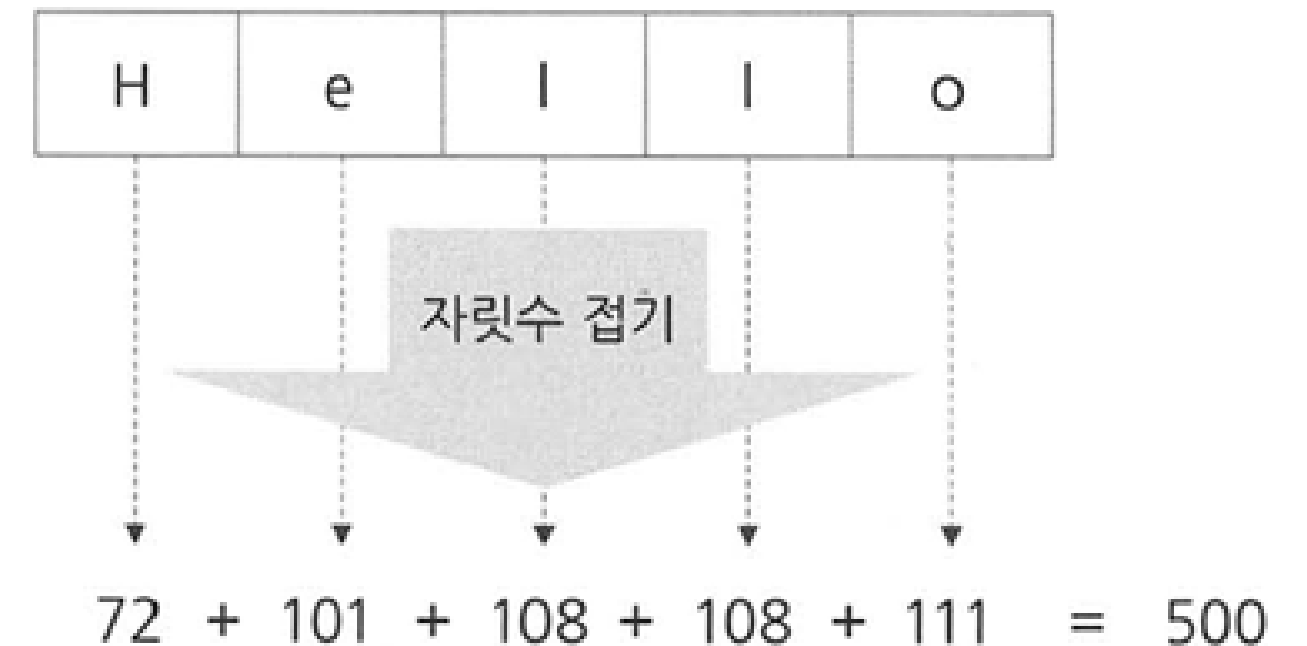
03

해시 함수의 종류

자릿수 접기

- 종이 접듯이 숫자를 일정 크기 이하의 수로 만드는 방법
- 키를 일정 자릿수로 나눠 합산 후 테이블 크기로 나눈 나머지를 반환
- 예) 452311607일 때
 - 한자리씩 접기: $4+5+2+3+1+1+6+0+7 = 29$
 - 두자리씩 접기: $4+52+31+16+07 = 110$
- 테이블 크기가 100이라면 $29 \rightarrow 29$, $110 \rightarrow 10$
- 9자리 십진수라면 한자리씩 접기 적용 시 해시 값: 최소 0, 최대 81
 - 두자리씩 접기 적용 시 해시 값: 최소 0, 최대 405
- 문자열을 키로 사용하는 해시 테이블에 많이 사용된다. 각 문자의 아스키코드 번호를 사용하고 각각 더하는 식으로 사용

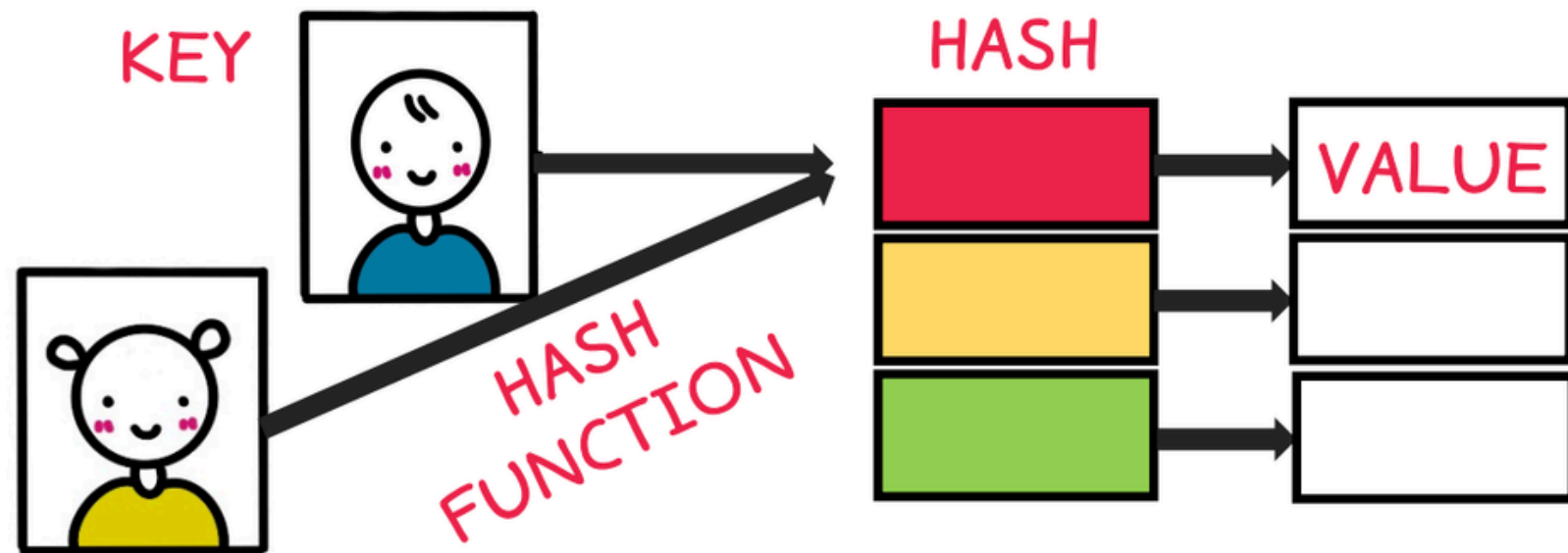
+ 중간 제공법, 곱셈법 등...



01

해싱과 충돌(Collision)

- 키 값이 다른데, 해시 함수의 결과값이 동일한 경우



- 충돌이란 해시 함수가 서로 다른 입력에 대해 동일한 해시 값을 반환하여 해시 테이블 내의 주소가 충돌하는 문제를 말한다.
- 즉, 서로 다른 키 값 x, y 가 있을 때, $h(x)=h(y)$ 인 경우를 의미한다.
- 서로 다른 키가 같은 인덱스로 매핑되는 상황
- 충돌 처리 없으면 데이터 손실 발생
- 아래 나눗셈법을 적용한 해시 테이블에 (30, 'cherry') 삽입하기

[해시 테이블 상태]

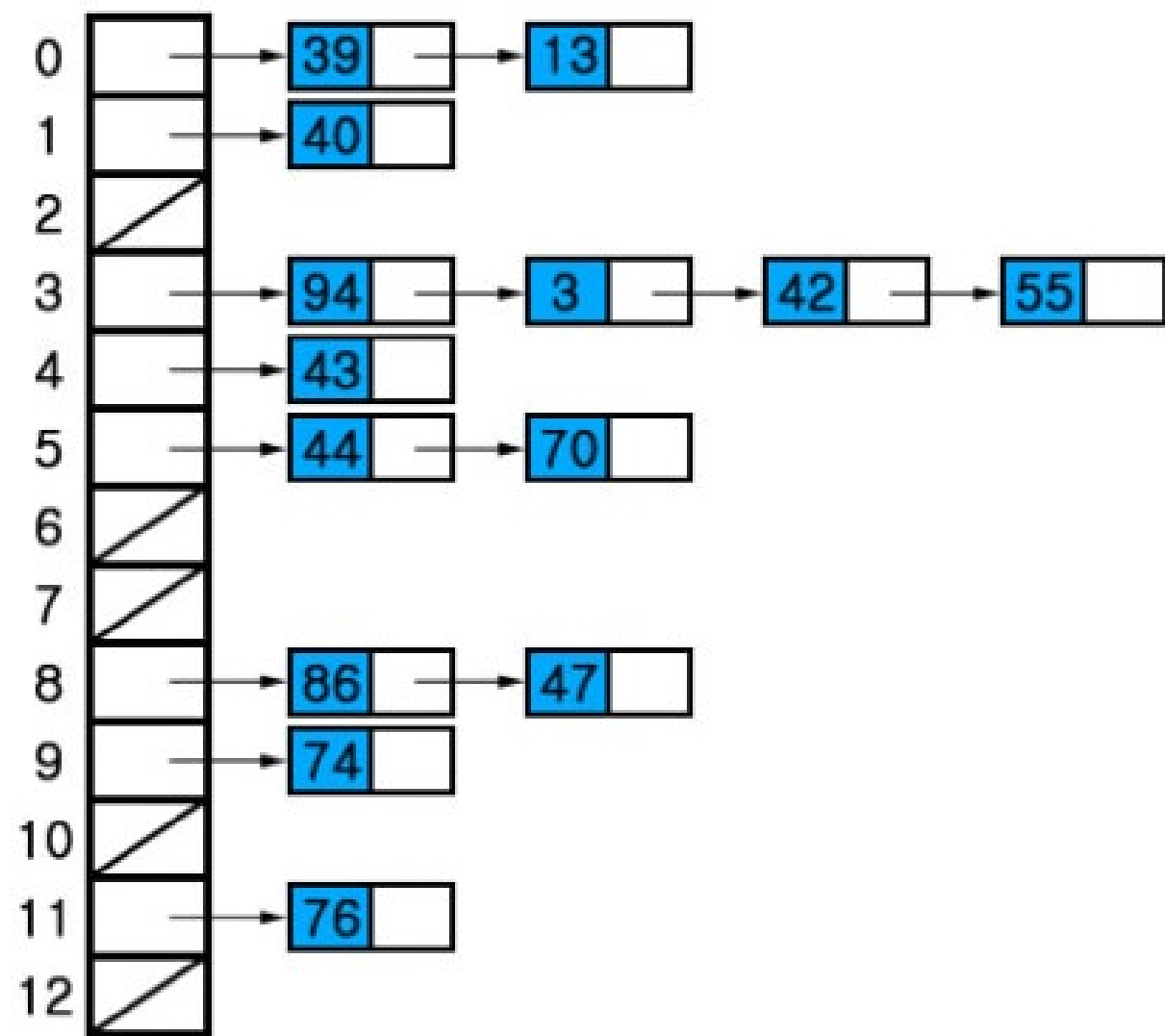
```
[0] None
[1] None
[2] (16, 'apple')
[3] None
[4] None
[5] (5, 'daisy')
[6] None
```

- 충돌을 해결하기 위한 방법으로 체이닝과 개방 주소법을 사용할 수 있다.

02

체이닝(Chaining)

사슬을 주렁주렁 엮는다



- 체이닝은 폐쇄 주소법, 개방 해싱 방법이다.
 - 폐쇄 주소법: 해시 함수에 의해 만들어진 주소만 사용하는 방법
 - 개방 해싱: 해시 테이블의 주소 바깥에 새로운 공간을 할당하여 문제를 수습
- 해시 함수가 서로 다른 키에 대해 같은 주소값을 반환해서 충돌 발생 시, 각 데이터를 해당 주소에 있는 링크드 리스트에 삽입하여 문제를 해결한다.
- 해시 테이블에는 데이터 대신 링크드 리스트에 대한 포인터를 관리한다.
- 탐색 연산: 해싱을 통해 주소값 찾기 → 해당 주소값에 연결된 링크드 리스트를 앞에서부터 뒤로 이동하며 목표값이 저장되어 있는지 비교
- 삭제 연산: 탐색과 같음
- 삽입 연산: 해싱을 통해 주소값 찾기 → 링크드 리스트가 비어 있으며 데이터 바로 삽입, 그렇지 않다면 **헤드 앞에 삽입**
- 체이닝의 성능 향상을 위해 이진탐색트리와 체이닝을 결합할 수 있다.

03

개방 주소법

금을 캐듯 메모리 공간을 탐사

- 개방 주소법: 충돌이 일어나면 해시 테이블 내의 새로운 주소를 탐사하여 충돌된 데이터를 입력한다.
- 폐쇄 해싱: 충돌 발생 시 테이블 내의 다른 빈 공간을 찾아 데이터를 저장한다.
- 탐사 방법에 따라 선형 탐사, 제곱 탐사, 이중 해싱, 재해싱 등의 종류가 있다.

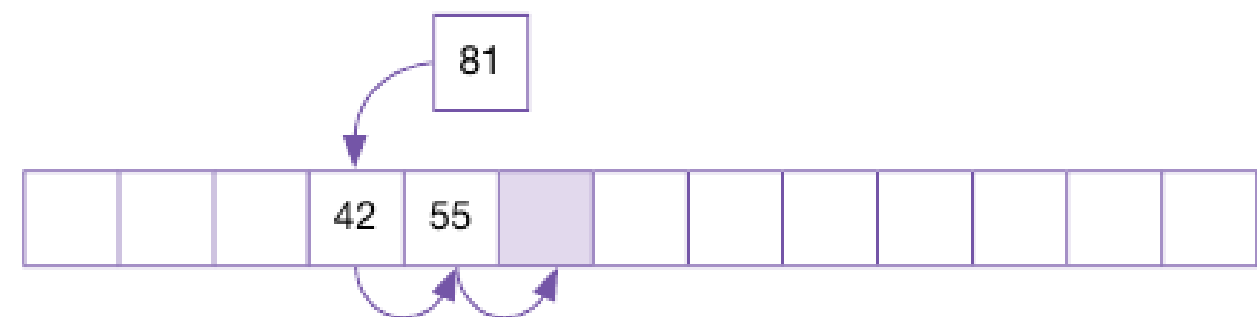
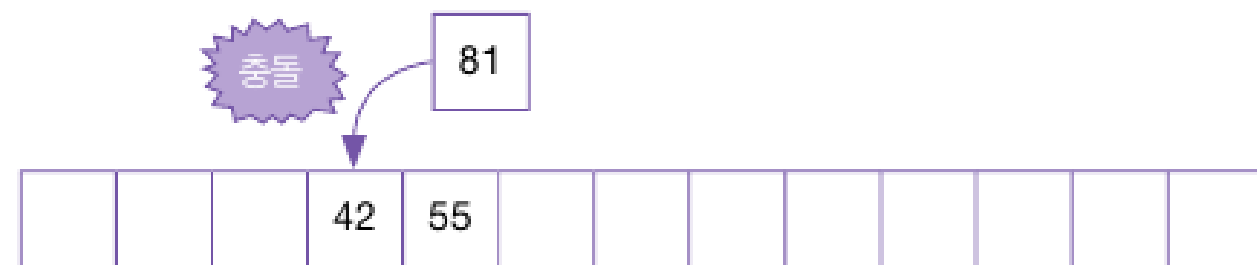
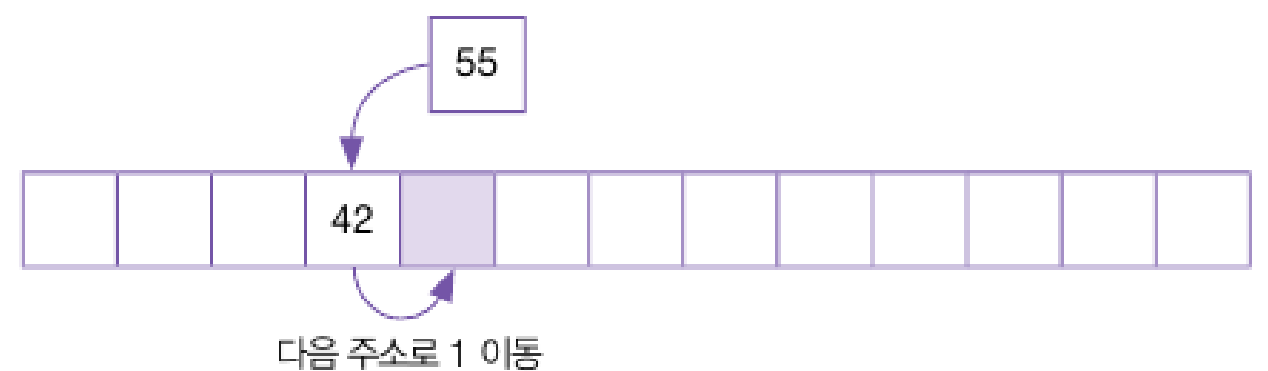
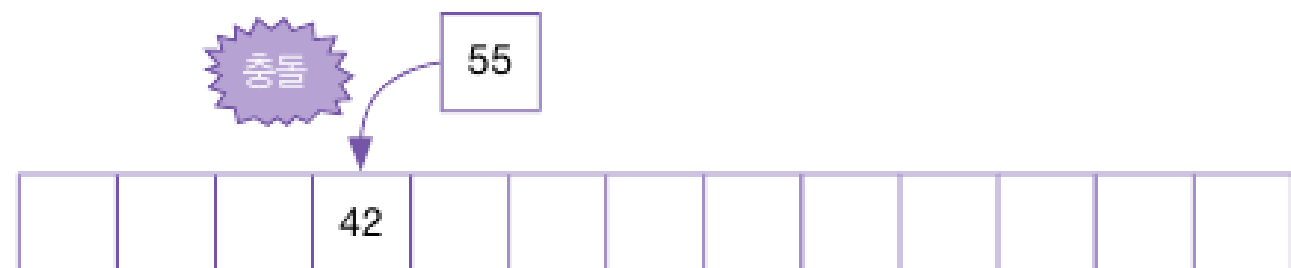
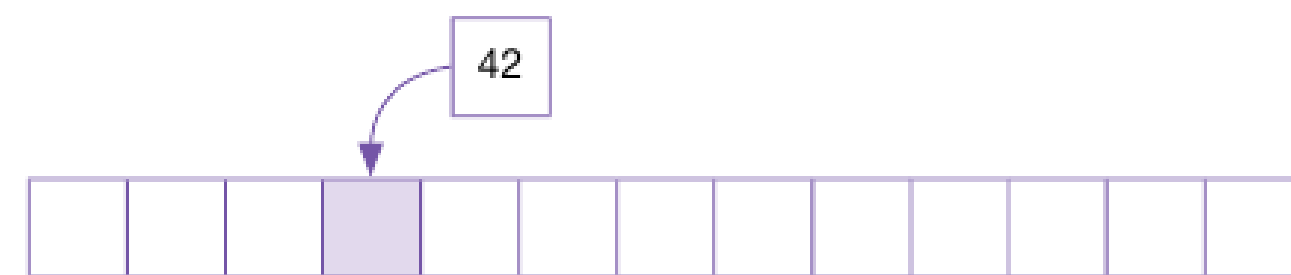


04

선형 탐사

빈 자리 찾을 때까지 성실히 하나씩 살펴보기

- 해시 함수로부터 얻어낸 주소에 이미 다른 데이터가 입력되어 있을 경우 현재 주소에서 고정 폭으로 주소를 이동한다.
- 그 주소에도 다른 데이터가 있어 충돌이 발생하면 또 그다음 주소로 이동
- 비어있는 주소를 찾아내면 그곳에 데이터 입력
- 선형 탐사의 문제점: **클러스터** 현상 - 충돌할 뻔한 데이터들이 한 곳에 모이는 현상

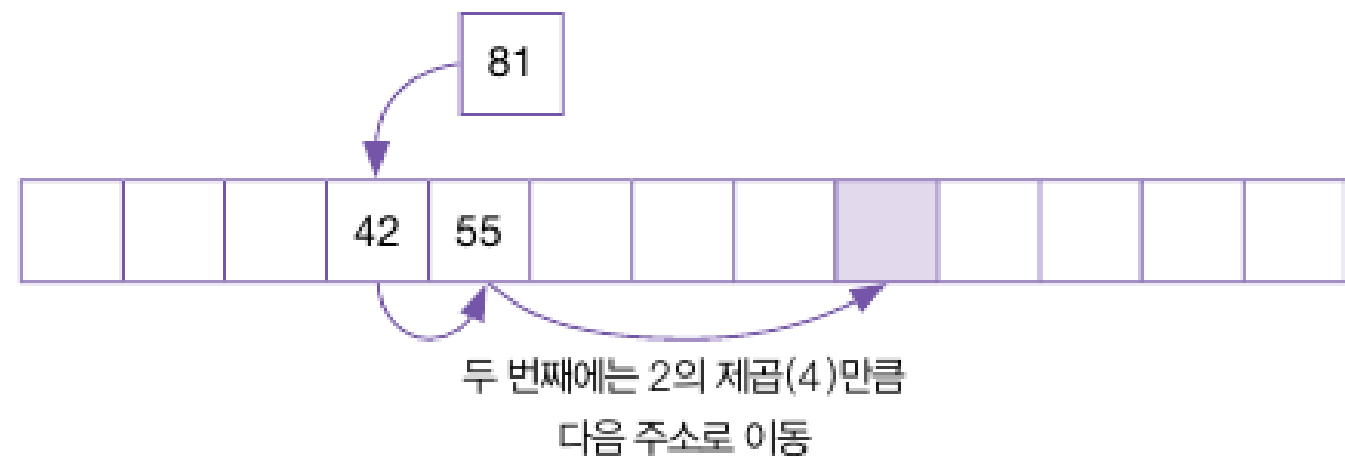
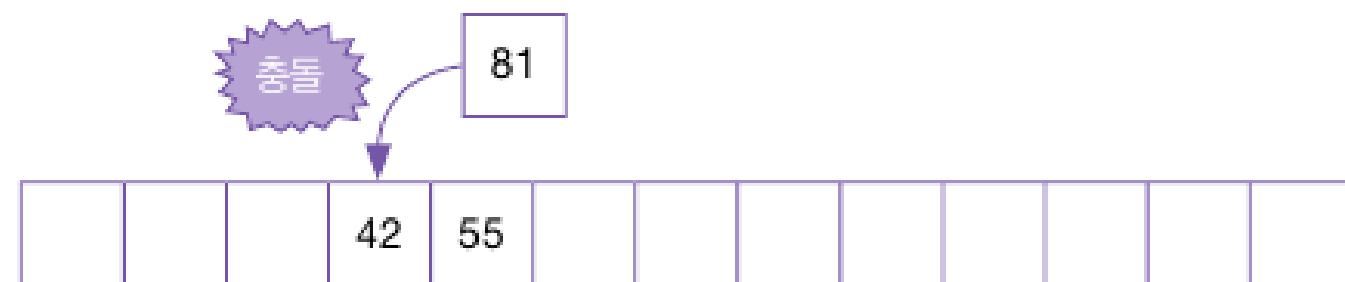
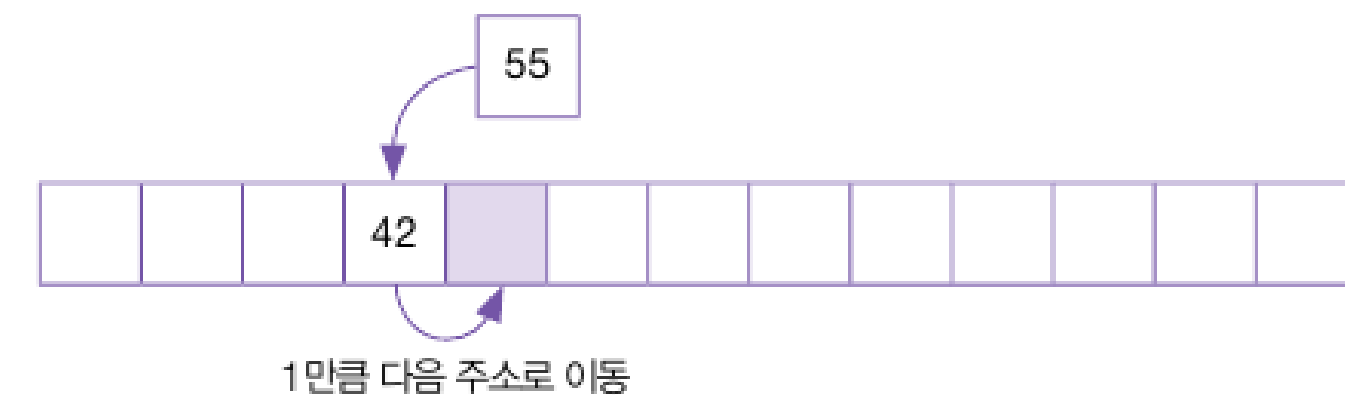
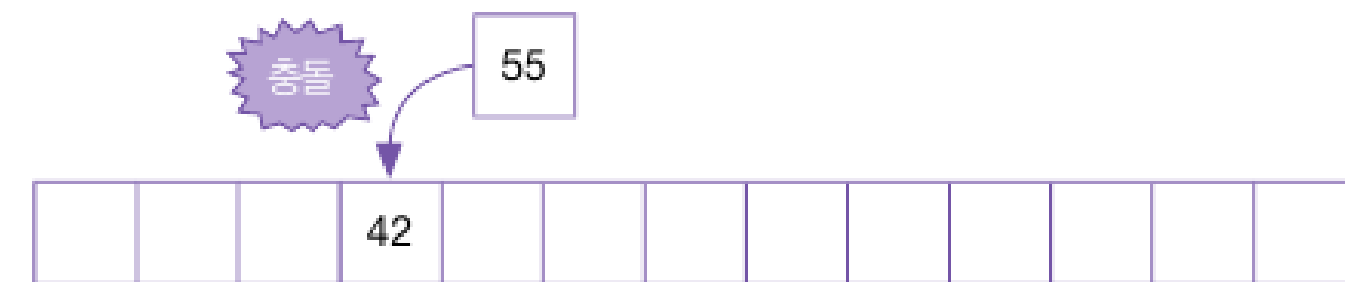
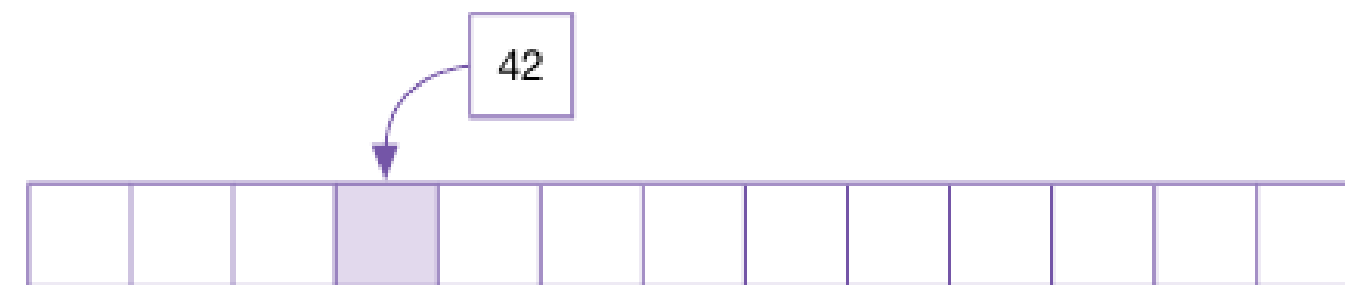
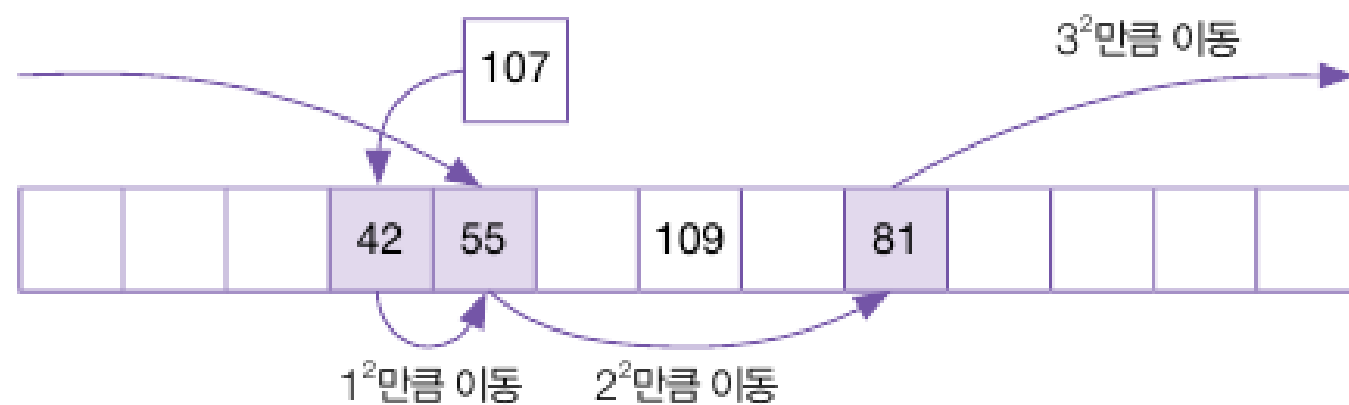


05

제공 탐사

약간 잔머리를 써보자

- 해시 함수로부터 얻어낸 주소에 이미 다른 데이터가 입력되어 있을 경우 현재 주소에서 제공수의 이동폭으로 주소를 이동한다.
- 선형 탐사와 비슷하지만, 이동폭이 고정이지 않고 제공수로 늘어난다.
- 규칙성에 의해 또 다른 종류의 클러스터가 발생한다는 문제점을 갖는다.
- 같은 해시값을 가진 데이터에 대해서는 2차 클러스터 발생



06

이중 해싱

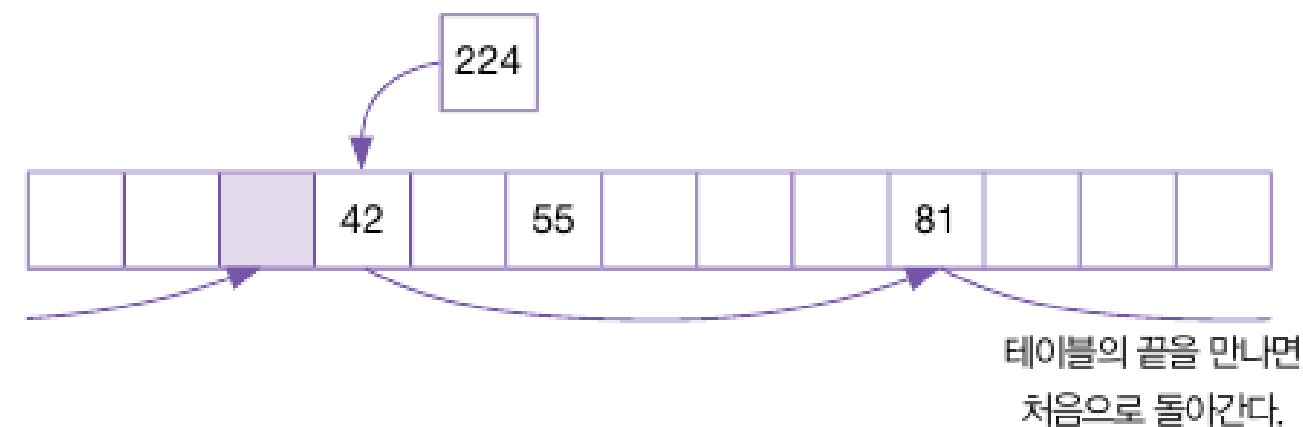
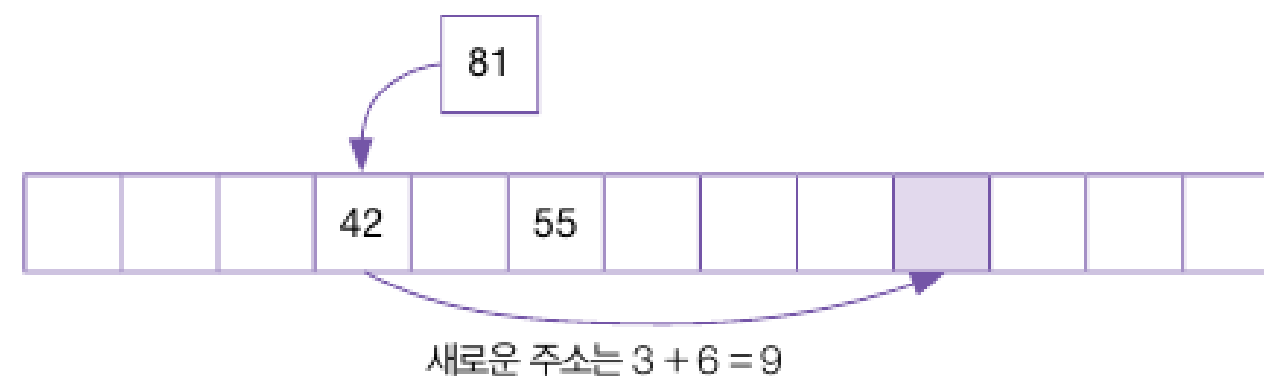
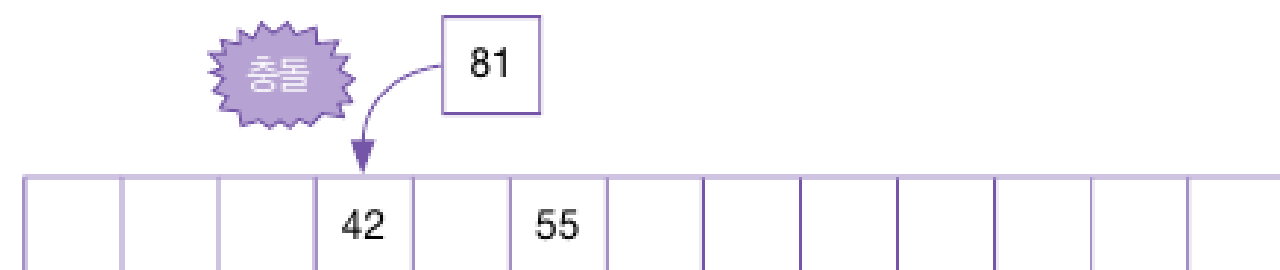
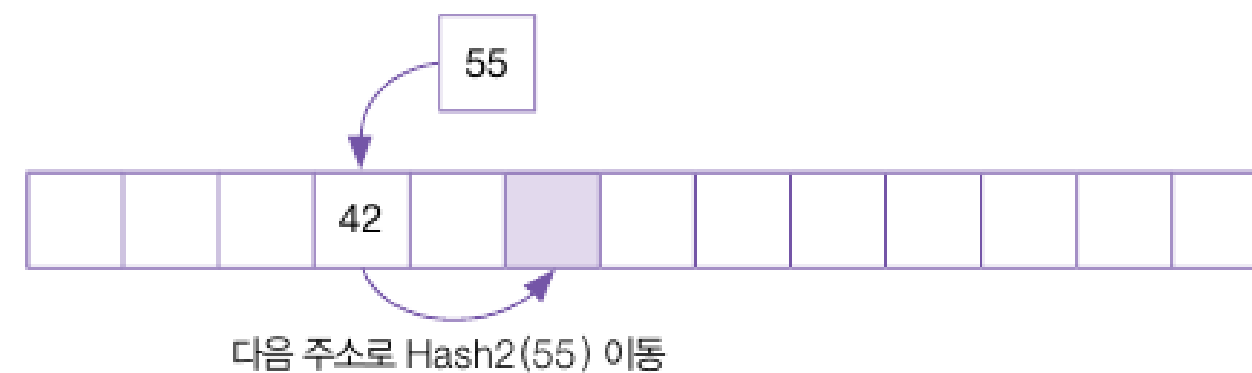
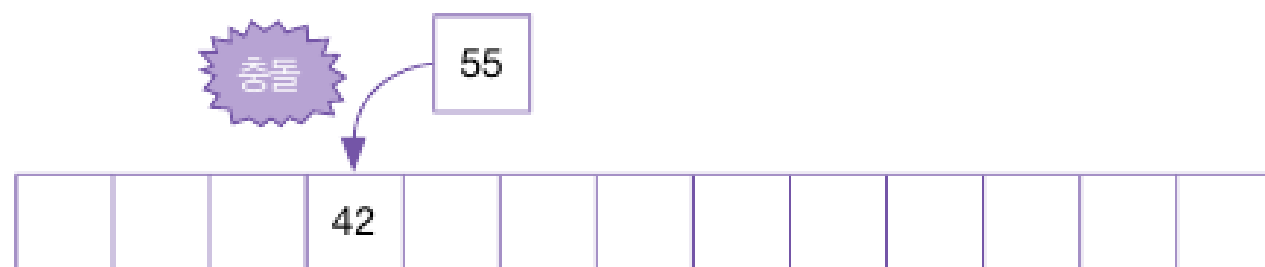
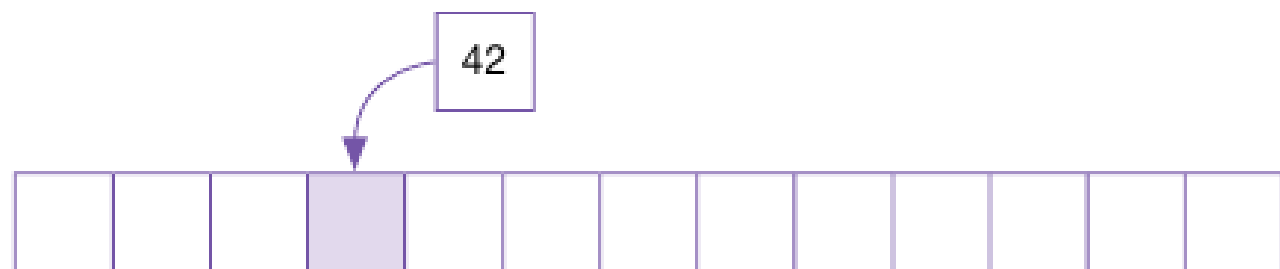
클러스터를 없애려면?

```

1 // 처음 해싱할 함수
2 int Hash(int key)
3 {
4     return Key % 13; // 테이블 사이즈 13
5 }
6
7 // 충돌 시 사용하는 함수
8 int Hash2(int Key)
9 {
10    return (Key % 11) + 2;
11 }

```

- 클러스터를 방지하기 위해 탐사할 주소의 규칙성을 없애고자 함.
- 이중 해싱은 이동폭을 제2의 해시 함수로 계산하는 방법
- 두 개의 해시 함수를 준비해서, 하나는 최소 주소 얻기, 또 다른 하나는 충돌 발생 시 이동폭 계산하기에 사용
- → 이동폭의 규칙성 제거

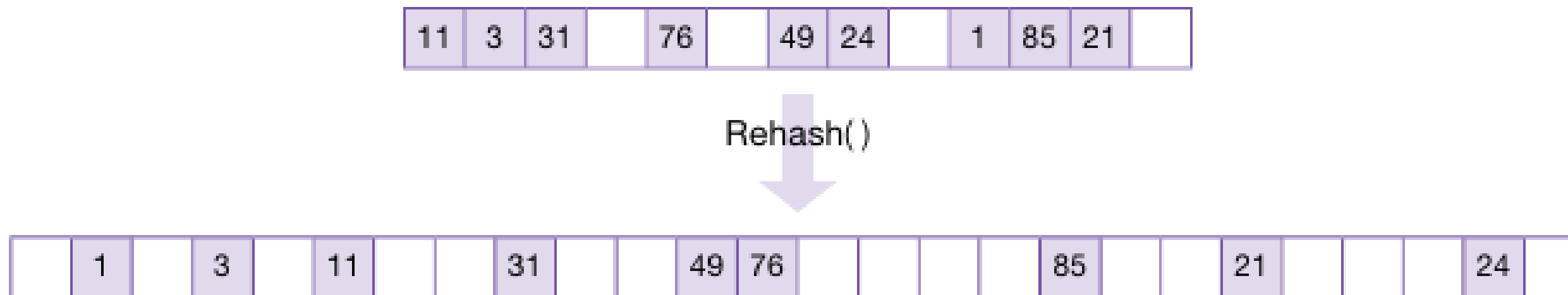


07

재해싱

꽤 찬 해시 테이블은 비효율적이다.

- 남은 공간이 거의 없는 해시 테이블에서는 연쇄 충돌이 자주 일어난다.
- 해시 테이블이 70~80% 차면 성능 저하가 나타나기 시작한다.
- 재해싱은 해시 테이블의 크기를 늘리고 늘어난 해시 테이블의 크기에 맞춰 테이블 내의 모든 데이터를 다시 해싱한다.
- 여유 공간을 늘리기 위한 작업
- 일반적으로 75% 임계치 설정



01

배운 내용 구현하기

- 자릿수 접기
- 체이닝
- BST + 체이닝
- 선형 탐사
- 제곱 탐사
- 이중 해싱
- 재해싱

02

급식실 출석 확인

A학교 급식실에서는 학생이 식권 단말기에 학번을 입력하면 이름을 조회해 출석을 확인한다. 점심시간 12시에 전교생이 한꺼번에 몰리기 때문에 조회 속도가 매우 중요하다. 학기 초 전교생 명단을 1회 등록해 놓으며, 매일 점심시간에 학생은 자신의 학번을 입력한다. 잘못된 학번이 입력되면 출입을 금지하며, 전학을 가게 될 경우 명단에서 삭제한다. 학번을 키(key), 이름을 값(value)으로 저장하는 해시 테이블을 구현하라.

(단, 3학년 4반 10번의 학번은 3410이다.)

03-A

두 개의 수로 특정값 만들기

n개의 양의 정수로 이루어진 리스트 arr와 정수 target이 주어졌을 때 이 중에서 합이 target인 두 수가 arr에 있는지 찾고, 있으면 True, 없으면 False를 반환하는 solution() 함수를 작성하세요.

[제약 조건]

n은 2 이상 10,000 이하의 자연수입니다.

arr의 각 원소는 1 이상 10,000 이하의 자연수입니다.

arr의 원소 중 중복되는 원소는 없습니다.

target은 1 이상 20,000 이하의 자연수입니다.

03-B

숫자 카드 개수 구하기

문제: 숫자 카드는 정수 하나가 적혀져 있는 카드이다. 상근이는 숫자 카드 N 개를 가지고 있다. 정수 M 개가 주어졌을 때, 이 수가 적혀있는 숫자 카드를 상근이가 몇 개 가지고 있는지 구하는 프로그램을 작성하시오.

입력: 첫째 줄에 상근이가 가지고 있는 숫자 카드의 개수 N ($1 \leq N \leq 500,000$)이 주어진다. 둘째 줄에는 숫자 카드에 적혀있는 정수가 주어진다. 숫자 카드에 적혀있는 수는 $-10,000,000$ 보다 크거나 같고, $10,000,000$ 보다 작거나 같다.

셋째 줄에는 M ($1 \leq M \leq 500,000$)이 주어진다. 넷째 줄에는 상근이가 몇 개 가지고 있는 숫자 카드인지 구해야 할 M 개의 정수가 주어지며, 이 수는 공백으로 구분되어져 있다. 이 수도 $-10,000,000$ 보다 크거나 같고, $10,000,000$ 보다 작거나 같다.

출력: 첫째 줄에 입력으로 주어진 M 개의 수에 대해서, 각 수가 적힌 숫자 카드를 상근이가 몇 개 가지고 있는지를 공백으로 구분해 출력한다.

01

자릿수 접기

- 아래 자리부터 끊을 경우

```
def hash_fn(self, key):  
    digits = str(key)  
    chunks = [digits[i:i+self.chunk_size] for i in range(0, len(digits), self.chunk_size)]  
    total = sum(int(chunk) for chunk in chunks)  
    return total % self.size
```

- 이동 폴딩과 경계 폴딩으로 나눌 수 있다.
 - 이동 폴딩: 키를 여러 부분으로 나눈 값들을 더한다.
 - 경계 폴딩: 키의 이웃한 부분을 거꾸로 더한다.

탐색키	123	203	241	112	20						
이동폴딩	123	+	203	+	241	+	112	+	20	=	699
경계폴딩	123	+	302	+	241	+	211	+	20	=	897

02

enumerate()

날날이 세다

- enumerate()는 인덱스와 원소로 이뤄진 튜플을 만든다.
- 시퀀스를 순회할 때 인덱스와 값을 함께 반환
- range 대신 사용 가능

```

1 fruits = ["apple", "banana", "cherry"]
2
3 for v in fruits:
4     print(v)
5
6 # enumerate → 인덱스 + 값 같이 나옴
7 for i, v in enumerate(fruits):
8     print(i, v)
9 # 0 apple
10 # 1 banana
11 # 2 cherry

```

```

def display(self):
    print("\n[ 해시 테이블 상태 ]")
    for i, slot in enumerate(self.table):
        print(f" [{i}] {slot}")
    print()

```

```

[ 해시 테이블 상태 ]
[0] None
[1] None
[2] (16, 'apple')
[3] None
[4] None
[5] (5, 'daisy')
[6] None

```

bucket

	slot		
	0	1	2
0	data	data	data
1	data		
2	data	data	
...	data		
10	data	data	data

인덱스 하나의 칸을 ‘버킷’이라 함.

03

체이닝

- 리스트로 체이닝할 경우

```
class HashTableDivision:
    def __init__(self, size=7):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_fn(self, key):
        return key % self.size

    def insert(self, key, value):
        idx = self.hash_fn(key)
        self.table[idx].insert(0, (key, value))
```

04

3-A 두 개의 수로 특정값 만들기

- set나 dictionary가 내부적으로 해시 테이블로 구현되어 있음.
- 직접 구현한 해시 테이블 사용해도 되지만, set나 dictionary 사용해도 삽입/탐색 속도 보장

05

3-B 숫자 카드 개수 구하기

- 딕셔너리를 만들고,
- 입력되는 카드 리스트를 바탕으로
- 숫자: 개수 쌍을 딕셔너리에 추가한다.

```
8
2 3 2 1 5 7 6 6
4
1 2 3 4
1 2 1 0
```


06

해싱의 응용 분야

- 인터넷 검색 엔진에서 널리 사용된다.
 - URL을 키, 페이지 내용을 값으로 해시 테이블에 저장
 - 수십억 개의 웹페이지 중 관련 페이지를 $O(1)$ 으로 즉시 조회
- 데이터베이스 인덱싱
 - 자주 조회되는 컬럼(학번, 주민번호 등)을 해시 테이블로 인덱싱
 - 테이블 전체 스캔 없이 레코드의 디스크 위치를 바로 조회
- 컴파일러의 심볼 테이블
 - 변수명/함수명을 키, 자료형/메모리 주소를 값으로 저장
 - 변수 참조 시 $O(1)$ 으로 조회
 - 선언하지 않은 변수 사용 시 MISS → 컴파일 에러 발생

Q&A
